SPRINT 3 · PROMPT & AI

ChargeGrid Intelligence

Do protótipo em Google Colab a uma aplicação local, modular, segura e avaliada

Autores: Luigi Freire · Victor Vidigal · Gabriel Savoy · Raphael Tien

627	1.964	65	10/10
linhas na célula principal do notebook	linhas de código Python modular	testes automatizados aprovados	casos aprovados no benchmark

Evolução em uma frase

Oprotótipo comprovava o conceito,mas misturava dados, instruções, integração com o modelo, dashboard e interface em uma única célula. A versão atual transforma essa prova de conceito em um pacote Python reproduzível, no qual dados e cálculos são validados antes da inferência, o modelo responde por uma chain LCEL e a saída passa por um contrato Pydantic.

Mudança central de responsabilidade

Legado	Evolução	Resultado atual
Prompt com dados + GPT extrai JSON	→	JSON validado + indicadores em Python
Modelo calcula e redige	→	Modelo interpreta fatos já calculados
Texto livre + histórico global	→	Schema Pydantic + sessão isolada
Teste manual no notebook	→	Pytest + Ruff + runner de avaliações

Impacto: menor dependência de comportamento probabilístico, falhas mais fáceis de localizar, interfaces desacopladas e execução local sem a API paga da OpenAI.

1 · Arquitetura e stack: antes → depois

Dimensão	Notebook legado	Projeto atual	Benefício
Execução	Colab + !pip	Pacote local / pyproject	Instalação reproduzível
LLM	GPT-4o-mini / OpenAI	Gemma 4 31B / Ollama Cloud	Sem custo da API OpenAI
Dados	Embutidos no prompt	JSON + Pydantic	Fonte única e validável
Indicadores	Parte inferida pelo LLM	Regras Python testadas	Menos erro aritmético
Chain	Funções manuais	prompt | llm | parser	Fluxo LCEL explícito
Resposta	Texto livre	Schema Pydantic v2	Contrato estável
Memória	Lista global	Session ID + 4.000 tokens	Isolamento e limite
Segurança	Apenas instruções	Guardrails antes do LLM	Recusa previsível
Interface	Loop + briefing	CLI + Gradio	Uso plug and play
Qualidade	Perguntas manuais	65 testes + 10 evals	Evidência repetível

Fluxo da aplicação atual

1. Entrada	2. Núcleo determinístico	3. IA e entrega
data/exemplo_operacao.json ↓ validação Pydantic	indicadores + regras ↓ contexto operacional	guardrails → memória → LCEL ↓ Gemma/Ollama → parser ↓ CLI ou Gradio

O dashboard Plotly usa os mesmos dados e indicadores validados. Ele não depende do texto produzido pelo modelo, evitando que uma variação de resposta altere os gráficos.

Stack consolidada

- Python3.11–3.14, LangChain/LCEL, langchain-ollama e Ollama Cloud.
- Pydantic v2 para entrada e saída; Plotly para análise visual; Gradio e CLI para acesso.
- Pytest e Ruff para verificação; tiktoken para estimativa comparável de tokens.

2 · Como os critérios da Sprint 3 foram aplicados

Critério	Aplicação no projeto	Como ajudou	Situação
A · LCEL	builder.py: prompt | llm | parser	Composição clara e testável	Aplicado
A · Memória	RunnableWithMessageHistory; sessão; 4k tokens	Continuidade sem mistura entre sessões	Aplicado*
A · Schema	Objetos aninhados, enums e validadores	Saída previsível para as interfaces	Aplicado
A · Contexto	JSON validado + indicadores pré-calculados	Reduz alucinação e erro de cálculo	Aplicado
B · Prompt	v1/v2, XML, VERSIONS e tiktoken	Rastreia decisões e custo	Aplicado
B · Modelos	Qwen indisponível; Gemma validado	Escolha compatível com free tier	Parcial**
C · Guardrails	Injection, escopo e risco elétrico	Bloqueio antes de consumir o LLM	Aplicado
D · Avaliação	10 casos + JSONs de resultados	Comparação reproduzível	Aplicado

Antes/depois mensurável: prompt v1 → prompt v2

Métrica obrigatória	v1	v2	Variação / leitura
Qualidade	100% · 10/10	100% · 10/10	Empate
Tokens entrada/turno	1.951,5	2.360,5	+21,0%
Tokens saída/turno	237,6	254,9	+7,3%
Latência média	1,273 s	3,011 s	+136,5%
Structured output	100% · 6/6	100% · 6/6	Contrato preservado
Aderência aos fatos	100% · 21/21	100% · 21/21	Sem regressão
Guardrails	100% · 4/4	100% · 4/4	Sem regressão

Interpretação: o v2 manteve todas as aprovações e explicitou fontes, segurança e hierarquia de instruções. O custo foi maior uso de contexto e latência. Como o v1 já atingiu o teto, os 10 casos atuais não demonstram ganho percentual de qualidade; uma suíte mais adversarial é necessária.

Nota metodológica: esta tabela compara dois prompts na arquitetura atual, com o mesmo modelo e casos. O notebook legado não possuía runner, schema ou telemetria equivalente e, por isso, não recebeu números retrospectivos inventados.

3 · Problemas encontrados, soluções e decisões

Problema no legado	Solução implementada	Por que foi escolhida
Dados presos ao prompt; outra chamada extraía JSON	Arquivo JSON + modelos Pydantic	Remove uma chamada e permite trocar/validar dados
LLM também fazia contas críticas	Indicadores e throttling em Python	Funções determinísticas são auditáveis e testáveis
Colab + chave OpenAI	Pacote local + Ollama Cloud	Reprodução local e aderência ao free tier
Saída textual variável	PydanticOutputParser + validadores	Falha controlada e contrato para UI
Histórico global sem limite	Session ID + corte por tokens	Isolamento e contexto previsível
Sem filtro antes do modelo	Guardrails determinísticos	Recusa consistente e economia de tokens
Briefing automático ao iniciar	Saudação local e neutra	Chat começa sem chamada e respeita a intenção do usuário

Trade-offs assumidos

- O prompt v2 documenta maisregras, mas aumentou tokens de entrada e latência no benchmark.
- O Ollama Cloud remove o custo direto da OpenAI, mas requer autenticação e conexão de rede.
- Schemas e camadas aumentam o volume de código, porém tornam cada falha localizável e coberta por testes. O
- limite de memória controla custo e contexto;
- Guardrails por padrões são rápidos e previsíveis, mas precisam crescer junto com novos ataques observados.

Evidência técnica

- 65 testes aprovados; Ruff sem erros.
- 10/10 casos de avaliação aprovados nas versões v1 e v2.

4 · Rastreabilidade, equipe e próximos passos

Integrante	RM	Contribuições
Luigi Freire	569958	IA, Prompts e Structured Output
Victor Vidigal	571318	Dados e Regras de Negócio
Gabriel Savoy	568991	Interfaces, visualização e documentação
Raphael Tien	570261	Memória, Guardrails e Avaliações

Próximos Passos

Prioridade	Ação	Critério beneficiado
1	Adicionar sumarização quando a memória exceder 4.000 tokens	A · Memória
2	Ampliar casos adversariais, ambíguos e conversas de 3+ turnos	C/D · Guardrails e avaliação
3	Tratar a depreciação da API de histórico	Manutenibilidade

Fontes auditadas

- notebooks/Chatbot_ChargeGrid_IA.ipynb — protótipo e stack legada.
- src/chain, src/schemas, src/guardrails e src/chargegrid.
- evals/baseline_results.json e evals/sprint3_results.json.
- Git log, README.md, pyproject.toml e suíte tests/ — autoria, instalação e verificação.

Conclusão: a evolução mais relevante não é apenas a troca do GPT pelo Gemma. O projeto passou a separar fatos, regras, segurança e geração de linguagem. Essa arquitetura permite explicar o comportamento, testar cada camada e trocar dados, modelo ou interface sem reescrever o sistema inteiro.